



プログラミング概論I (オンデマンド授業)



担当: 中山 功一 (7号館3階308室)

メールアドレス: knakayama@is.saga-u.ac.jp

392

オープニングテスト

前回, 習ったことを復習するために,
オープニングテストを行います

問題ごとに動画を止めて, 答えをメモしましょう.

最後に, 答えを動画で表示しますので,
自分が理解しているか, 確認しましょう.

オープニング テスト

右のように使う関数のうち,
【問1】戻り値がある関数は?
【問2】引数がある関数は?

あ: `x=F(a,b);`
い: `x=F();`
う: `F(a,b);`
え: `F();`

<選択肢>

- ①あ／い, ②あ／う, ③あ／え
④い／う, ⑤い／え, ⑥う／え
⑦その他

394

オープニング テスト

【問4】

【問6】

`int max( int array[10]);`

【問3】

【問5】

<選択肢>

- ①引数の型 ②引数の名前
③戻り値の型 ④戻り値の名前
⑤関数の型 ⑥関数の名前

395

オープニング テスト

以下の関数のプロトタイプ宣言を書きなさい
(空白および省略可能なvoidは省略)

－ 関数名はMix, 戻り値はint, 仮引数は無し

【問7】

－ 仮引数はdouble dataとint num, 戻り値は無し,
関数名はMix

【問8】

<選択肢> ①double, ②int, ③ num , ④ data,
⑤void, ⑥Mix, ⑦(, ⑧), ⑨【, (コンマ)】, ⑩ ;

396

オープニングテスト

- 問9: 定義した関数内のみでアクセスできる変数
 問10: プログラム内のどの関数からでも
 アクセスできる変数
 問11: 上記のうち、通常、用いるべき変数
 問12: 関数を呼び出すごとに初期化される変数
 問13: 一度呼び出すとプログラム終了まで
 値が保持される変数
 問14: 上記のうち、通常、用いるべき変数
 <選択肢> ①静的変数, ②大域変数,
 ③動変数, ④局所変数

397

答え合わせ

全ての答えをメモしましたか？

解答

- | | |
|-----------------------------------|--------|
| 【1】 ① | 【9】 ④ |
| 【2】 ② | 【10】 ② |
| 【3】 ③ | 【11】 ④ |
| 【4】 ⑥ | 【12】 ③ |
| 【5】 ① | 【13】 ① |
| 【6】 ② | 【14】 ③ |
| 【7】 ②⑥⑦⑧⑩ int Mix(); | |
| 【8】 ⑤⑥⑦①④⑨②③⑧⑩ | |
| void Mix (double data, int num); | |

各試験の難易度

<プログラミング演習支援システム>

<https://progs.v.is.saga-u.ac.jp/>



プログラミング概論: 自習用



実技試験の過去問



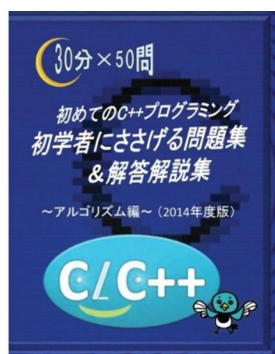
概論1 第1中間/第2中間の全て



例年、
正解数に応じて
挽回
加点
します

400

問題集 & 解答解説集 (紙版)



デザインエッグ社 中山功一著

Amazonでのみ販売
 (生協では売ってません)

過去の試験問題や
 今後の試験問題の
 傾向の参考に

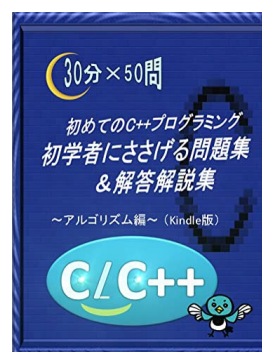
¥ 1,331



<https://www.amazon.co.jp/dp/4865430512/>

401

問題集 & 解答解説集 (kindle版)



2014年度版と
 内容は同じですが、
 コピペができます

※ Kindle Unlimited 対応

あと、安いです。

¥ 500



<https://www.amazon.co.jp/dp/B09ZXP9N3J/>

402

過去問10年分: 解答解説付 (kindle版)



過去問10年分
(解答例と解説付き)
・コピーができます
※問題はプログラミング
演習支援システムと同じ
※ Kindle Unlimited 対応

¥ 1,000
(約2,000ページ)
※毎年, 増加中



<https://www.amazon.co.jp/dp/B09ZVG5Z4Q/>

403

ミス指摘加点

問題集／授業スライド／試験問題／
プログラミング演習支援システムなどで,
改善に値する誤植／記載ミス／間違いを
指摘してくれた場合, 素点に1点ずつ加点

404

系統的なプログラミング手法

系統的なプログラミング手法

プログ概論: 基本知識

プログ演習: 実習

Step 1

問題を
理解

Step 2

解法を
アルゴ
リズム
で表現

Step 3

ソース
コード
に置き
換え

Step 4

ソース
をコン
パイル

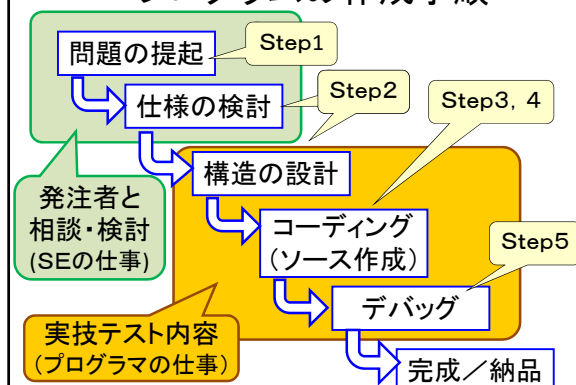
Step 5

修正
(デバ
ッグ)

アルゴリズム: 解決手順を日本語で表現したもの
コンパイル: ソースを実行ファイルに変換すること

405

プログラムの作成手順



プログラム構造の設計手順

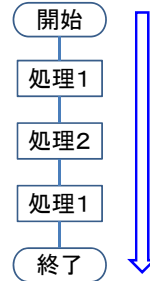
- (1) アルゴリズム構造を段階的に考える
 - 大きな作業の塊(≡関数)に分割
(基本設計／外部設計)
 - さらに単純な作業(≡数行)に分割
(詳細設計／内部設計)
- (2) 各作業に必要なデータ構造を考える
 - 各作業に必要な変数を決める
- (3) 組み合わせて確認する

アルゴリズム構造の基本

- 開始場所(1ヶ所)
- 何らかの処理
 - 処理するデータ(data[n] = {}など)
 - または、入力(cin, file.getなど)
 - 出力(coutなど) 例外あり
- 終了場所(通常は1ヶ所)

チャートによる表現: 順次(逐次)

• フローチャート



• NSチャート



アルゴリズムのチャート表現

• フローチャート

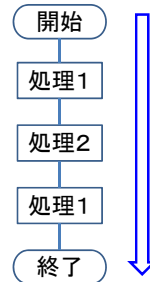
- 最も古く、一般的に利用されている
- △ いくつかの流派がある(書き方が微妙に違う)
- ✗ 分かりにくいアルゴリズムでも書けてしまう
(=自由度が**高い**)

• NSチャート(構造化チャート)

- 書きやすい, 見やすい
- 構造化アルゴリズム専用(分かりやすい)
(=自由度が**低い**)

チャートによる表現: 順次(逐次)

• フローチャート

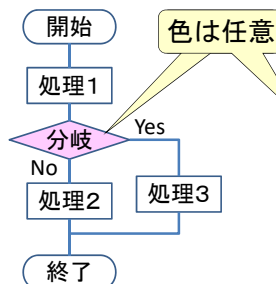


• NSチャート

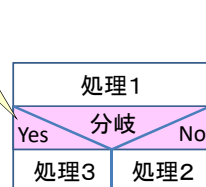


チャートによる表現: 分岐

• フローチャート

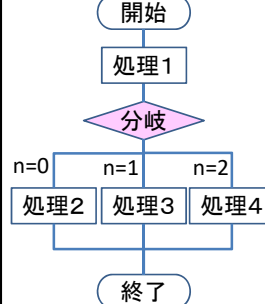


• NSチャート

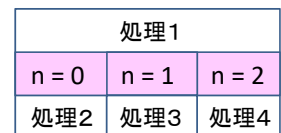


チャートによる表現: 多岐条件

• フローチャート

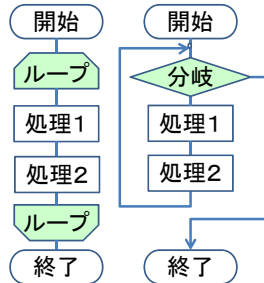


• NSチャート



チャートによる表現: ループ

• フローチャート



• NSチャート



NSチャート: ループの終わり

```

for(i=0;i<10;i++){
  処理1
  処理2
}

```



```

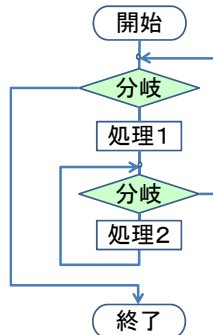
for(i=0;i<10;i++){
  処理1
}
処理2

```



チャートによる表現: 二重ループ

• フローチャート



• NSチャート



NSチャート: 二重ループの終わり

```

for(i=0;i<10;i++){
  for(j=0;j<10;j++){
    処理1
  }
  処理2
}

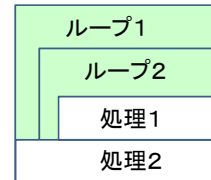
```



```

for(i=0;i<10;i++){
  for(j=0;j<10;j++){
    処理1
  }
}
処理2

```



アルゴリズム表現方法

• フローチャート

– 知識としては知っておいた方がよい

• NSチャート(構造化チャート)

– プログラム設計にはこちらがお勧め

アルゴリズムのチャート表現

• フローチャート

- ⊕ 最も古く、一般的に利用されている
- △ いくつかの流派がある(書き方が微妙に違う)
- ✗ 分かりにくいアルゴリズムでも書けてしまう(=自由度が**高い**)

• NSチャート(構造化チャート)

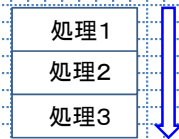
- ⊕ 書きやすい, 見やすい
- ⊕ 構造化アルゴリズム専用(分かりやすい)(=自由度が**低い**)

チャートによる表現: 順次(逐次)

・フローチャート

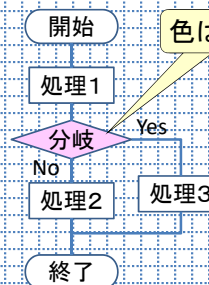


・NSチャート



チャートによる表現: 分岐

・フローチャート

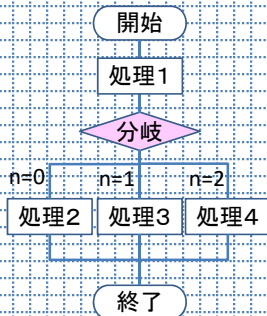


・NSチャート

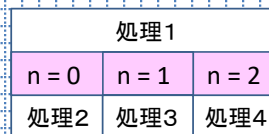


チャートによる表現: 多岐条件

・フローチャート

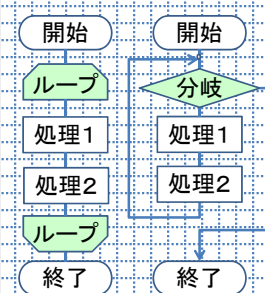


・NSチャート

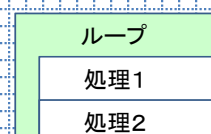


チャートによる表現: ループ

・フローチャート

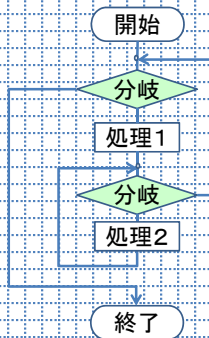


・NSチャート

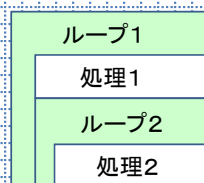


チャートによる表現: 二重ループ

・フローチャート



・NSチャート



アルゴリズム表現方法

・フローチャート

ー知識としては知っておいた方が良い

・NSチャート(構造化チャート)

ープログラム設計にはこちらがお勧め

問題

ここにN人の学生がいます
各人の身長／体重のデータから,
「平均身長」,「平均体重」,
「最大身長」,「最大体重」
を求めなさい

N人の身長・体重の平均・最大値

まずは日本語で
設計してみよう

N人の身長・体重の平均・最大値

最初にデータ構造を考えてみる

データ構造を考える(まずは図示)

- 身長:N人分

0	1	2	...	N-1
---	---	---	-----	-----
- 身長の平均値

Ave_Hei

- 身長の最大値

Max_Hei

- 体重:N人分

0	1	2	...	N-1
---	---	---	-----	-----
- 体重の平均値

Ave_Wei

- 体重の最大値

Max_Wei



変数定義を考える

データ構造を考える

- 身長:N人分 `double Hei[N];`
- 身長の平均値 `double Ave_Hei;`
- 身長の最大値 `double Max_Hei;`
- 体重:N人分 `double Wei[N];`
- 体重の平均値 `double Ave_Wei;`
- 体重の最大値 `double Max_Wei;`

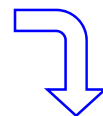


アルゴリズムを考える

アルゴリズムを考える

最初はざっくりと作業を分割

平均身長を求める
平均体重を求める
最大身長を求める
最大体重を求める
結果を表示する



続いて、個々の作業内容を検討
(図で動かしてみる)

最大身長を求める

- 身長: N人分

i番目が最大値なら
Max_Heiに代入

- 身長の最大値

0 1 2 ... N-1

↑ ↑ ↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

0 1 2 ... N-1

↑ ↑ ↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

0 1 2 ... N-1

↑ ↑ ↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

0 1 2 ... N-1

↑ ↑ ↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

0 1 2 ... N-1

↑ ↑ ↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

0 1 2 ... N-1

↑ ↑ ↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

0 1 2 ... N-1

↑ ↑ ↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

0 1 2 ... N-1

↑ ↑ ↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

↑

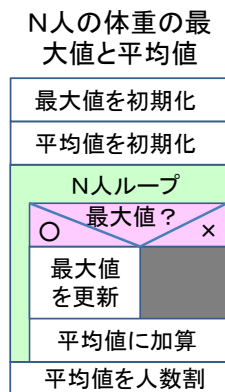
↑

↑

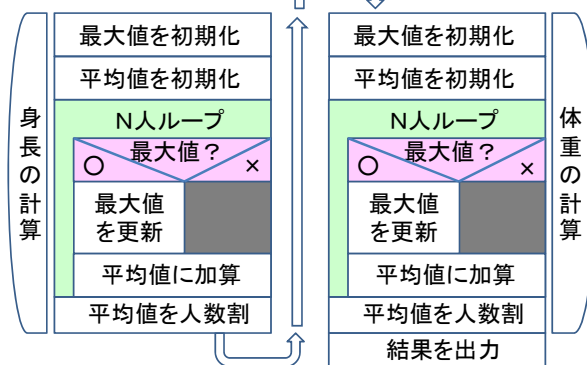
↑

↑

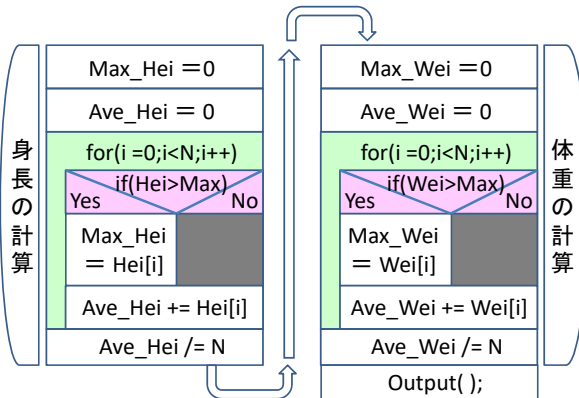
初心者は
日本語の時点で
アルゴリズムを
検討すること



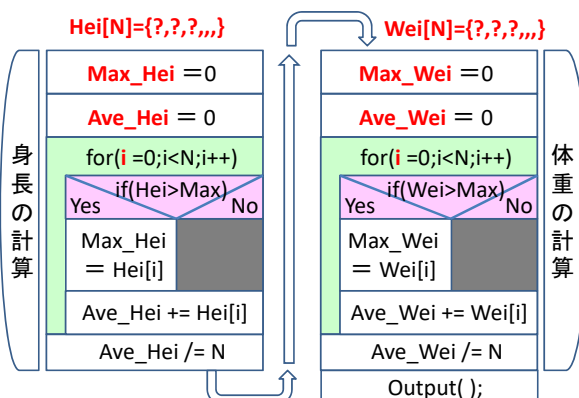
全体を統合(例)



C言語にするのは最後で良い



必要な変数を抜き出そう



現段階では、ここまでできれば、
プログラムの大部分は完了

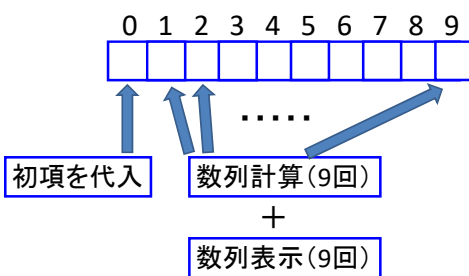
NSチャートを作ろう！その1

数列の計算

2011年度の間中テスト

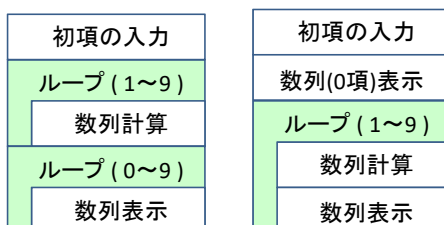
【問3:学籍番号_03_sequence.cpp】(20点)
 実数の数列「 $N_n = (N_{n-1} + 3.0) / 2.0$ 」(前項に3を足して2で割った値を次の項とする数列)を考えます
 1: double型1次元配列double sequence[10]を定義する
 2: 初項(第0項) $S_0 = \text{sequence}[0]$ にユーザが入力した実数を代入する
 3: 第1項から第9項までを、それぞれ前項を用いて計算し、 S_n の値をsequence[n]に代入する
 4: 数列sequence[0]～sequence[9]の値を、改行しながら順番に画面に表示する
 注: forループを用いることで、数列の項数(10)を容易に変更できるアルゴリズムとすること

ヒント



正解は・・・？

その1の正解例



```
void main(){
    double sequence[10];
    cout << "初項を入力してください" << endl;
    cin >> sequence[0];
    for(int n=1; n < 10; n++){
        sequence[n] = (sequence[n-1]+3.0)/2.0;
    }
    for(int n=0; n < 10; n++){
        cout << "第" << n << "項の値は" <<
            sequence[n] << "です" << endl;
    }
}
```

NSチャートを作ろう！その2

行列の加算と減算

3×3行列の和と差

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{matrix} + \\ - \end{matrix} \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix}$$

$$= \begin{bmatrix} a_{11} \pm b_{11} & a_{12} \pm b_{12} & a_{13} \pm b_{13} \\ a_{21} \pm b_{21} & a_{22} \pm b_{22} & a_{23} \pm b_{23} \\ a_{31} \pm b_{31} & a_{32} \pm b_{32} & a_{33} \pm b_{33} \end{bmatrix}$$

用意するデータ構造は？

452

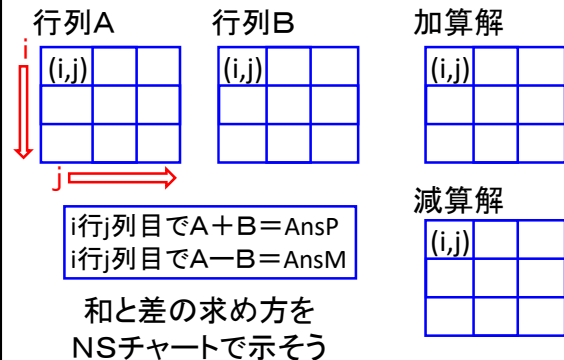
用意するデータ構造は？

- 問題の行列A : 配列a[3][3]
- 問題の行列B : 配列b[3][3]
- 加算の回答 : 配列AnsP[3][3]
- 減算の回答 : 配列AnsM[3][3]

データ構造を図で表そう

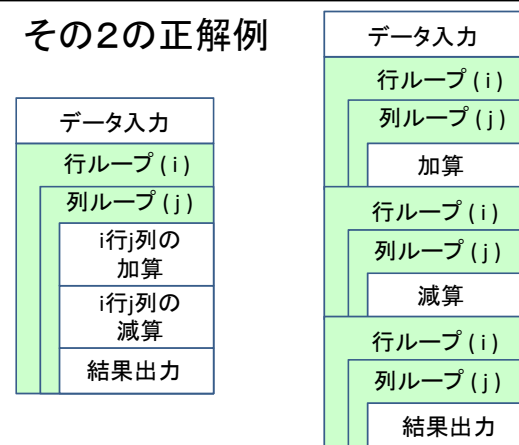
453

データ構造を図で表そう



正解は・・・？

その2の正解例



NSチャートを作ろう！その3

行列の掛け算

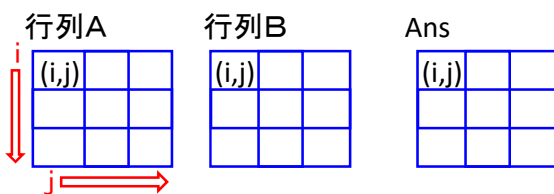
3×3行列の積

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11}+a_{12}b_{21}+a_{13}b_{31} & a_{11}b_{12}+a_{12}b_{22}+a_{13}b_{32} & a_{11}b_{13}+a_{12}b_{23}+a_{13}b_{33} \\ a_{21}b_{11}+a_{22}b_{21}+a_{23}b_{31} & a_{21}b_{12}+a_{22}b_{22}+a_{23}b_{32} & a_{21}b_{13}+a_{22}b_{23}+a_{23}b_{33} \\ a_{31}b_{11}+a_{32}b_{21}+a_{33}b_{31} & a_{31}b_{12}+a_{32}b_{22}+a_{33}b_{32} & a_{31}b_{13}+a_{32}b_{23}+a_{33}b_{33} \end{bmatrix}$$

458

用意するデータ構造は？

- 問題の行列A : 配列a[3][3]
- 問題の行列B : 配列b[3][3]
- 掛け算の回答 : 配列Ans[3][3]



掛け算の計算手順を考えよう

459

3×3行列の積(アニメーション)

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11}+a_{12}b_{21}+a_{13}b_{31} & a_{11}b_{12}+a_{12}b_{22}+a_{13}b_{32} & a_{11}b_{13}+a_{12}b_{23}+a_{13}b_{33} \\ a_{21}b_{11}+a_{22}b_{21}+a_{23}b_{31} & a_{21}b_{12}+a_{22}b_{22}+a_{23}b_{32} & a_{21}b_{13}+a_{22}b_{23}+a_{23}b_{33} \\ a_{31}b_{11}+a_{32}b_{21}+a_{33}b_{31} & a_{31}b_{12}+a_{32}b_{22}+a_{33}b_{32} & a_{31}b_{13}+a_{32}b_{23}+a_{33}b_{33} \end{bmatrix}$$

まず、各マス内部の計算は後回しにして考える

460

3×3行列の積

計算手順: 1行1列目を計算

$$a_{11} \times b_{11} + a_{12} \times b_{21} + a_{13} \times b_{31}$$

→ 1行2列目を計算 → 1行3列目,

→ 2行1列目 → 2行2列目 → 2行3列目

→ 3行1列目 → 3行2列目 → 3行3列目

i行j列目の掛け算の計算は以下で示す

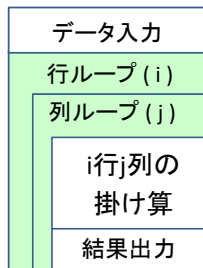
i行j列の
掛け算

この場合の、NSチャートを書きましょう

461

正解は・・・？

3 × 3行列の積



i行j列の掛け算の計算

1行1列目: $a_{11}b_{11}+a_{12}b_{21}+a_{13}b_{31}$

1行2列目: $a_{11}b_{12}+a_{12}b_{22}+a_{13}b_{32}$

2行1列目: $a_{21}b_{11}+a_{22}b_{21}+a_{23}b_{31}$

問題:i行j列目は?

$$a_{??}b_{??}+a_{??}b_{??}+a_{??}b_{??}$$

454

正解は・・・?

i行j列の掛け算の計算

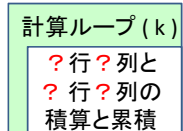
i行j列目の計算は

$$a_{i1}b_{1j}+a_{i2}b_{2j}+a_{i3}b_{3j}$$

kをループ変数とすると $a_{ik}b_{kj}$ と表現できる

問題:

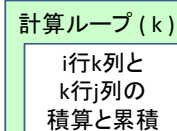
右図のNSチャートの
?を書きなさい



456

正解は・・・?

i行j列の掛け算の計算

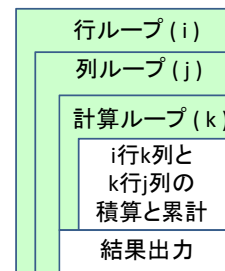


問題

これらを組み合わせて,
NSチャートで完成させなさい

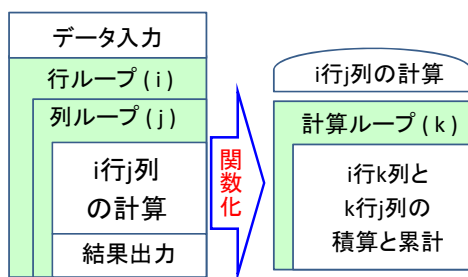
正解は・・・？

正解例その1



※データ入力省略

正解例その2

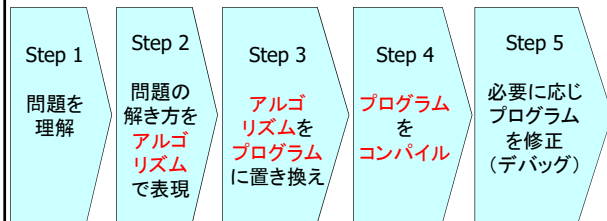


※データ入力省略

系統的なプログラミング手法

プログラミング概論: 基本知識

プログラミング演習: 実習



アルゴリズム: 問題を解く手順を日本語で表現したもの
コンパイル: プログラムを実行ファイルに変換すること

472

プログラム作成の系統的な手順(その1)

- **ステップ1** プログラムの設計と検査
 - 入出力を明確にする
 - プログラムに名前をつける
 - アルゴリズムを日本語で記述する
 - アルゴリズムに必要なデータを定義する
 - トレース表を作成して確認する

473

プログラム作成の系統的な手順(その2)

- **ステップ2** プログラムのコーディング
 - プログラムの宣言部を書く
 - アルゴリズムを高水準コメントで記述
 - コメントの下に、対応するコードを書いていく
 - コメントとコードが一貫しているか確認する

474

プログラム作成の系統的な手順(その3)

• ステップ3 コードの検査

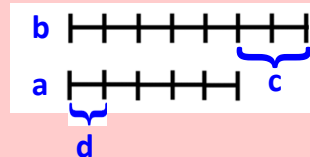
- プログラムを机上で実行する
(他人に動作を説明する)
- コンパイルし、エラーを修正する
- プログラムを実行してテストする
- デバッグを行なう

475

ユークリッド互除法: 最大公約数を求める方法

自然数 a, b ($a < b$) がある

b を a で割った余りを c とすると,
「 a と b の最大公約数 d 」と
「 a と c の最大公約数 d 」は等しい



476

ユークリッド互除法

c に, b を a で割った余りを代入

$$c = b \% a;$$

「 a と b の最大公約数」

= 「 a と c の最大公約数」

b に a を代入, a に c を代入

$$b = a; a = c;$$

これを繰り返し, 余りが0の時の
除数が最大公約数

477

Step 1: 問題を理解

入力された2つの整数値 A, B ($A < B$) の
最大公約数を求めよ。

(例題)

- 63と27の最大公約数を求める
- 63を27で割った余りは9
- 27を9で割った余りは0
- よって、最大公約数は9

必要な変数は何個？

478

Step 1: 問題を理解

変数 B (大), 変数 A (小), 変数 C (剰余, 最小)
 B と A の計算から, 次は A と C の計算になる。

変数を入れ替えると,

$$B = A; \text{//}(A \rightarrow B)$$

$$A = C; \text{//}(C \rightarrow A)$$

また変数 B と変数 A で計算できる

→ 同じことの“繰り返し”

割り切れたら終了(それが最大公約数)

479

Step 2: アルゴリズムの作成

ユークリッドの互除法

1. 整数値 A と B を入力する
2. $A > 0$ である限り以下の処理を繰り返す
 - 2.1 $B \% A$ の値を C とする
 - 2.2 A の値を B に代入する
 - 2.3 C の値を A に代入する
3. B の値を表示する



ユークリッド

480

Step 2: アルゴリズムの記述

ユークリッドの互除法

1. 整数値AとBを入力する
2. $A > 0$ である限り以下の処理を繰り返す
 - 2.1 $B \% A$ の値をCとする
 - 2.2 Aの値をBに代入する
 - 2.3 Cの値をAに代入する
3. Bの値を表示する

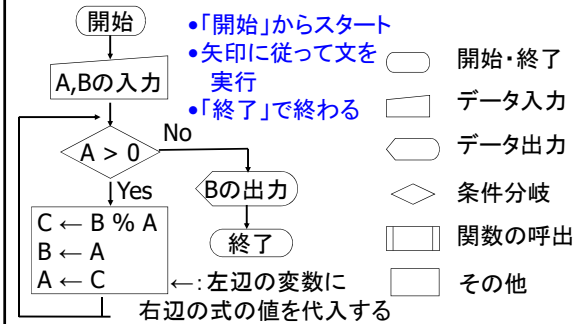
	A	B	C
1	27	63	-
2	27	63	-
2.1	27	63	9
2.2	27	27	9
2.3	9	27	9
2	9	27	9
2.1	9	27	0
2.2	9	9	0
2.3	0	9	0
2	0	9	0
3.			「9」を出力



ユークリッド

481

アルゴリズムの記述(フローチャートの例)



482

Step 2-1: トレース表の作成

- どのステップがどのような順序で実行されるか?
- どのような変数が使われるか?
- 各変数の値はステップ毎にどのように変化するか?

変数: アルゴリズムやプログラムが必要とするデータを格納するための場所

ステップ	A	B	C	
1	27	63	-	入力値
2	27	63	-	
2.1	27	63	9	
2.2	27	27	9	出力値
2.3	9	27	9	
2	9	27	9	
2.1	9	27	0	
2.2	9	9	0	
2.3	0	9	0	
2	0	9	0	
3	0	9	0	

483

Step 3: プログラミング

- プログラムのソースコードを実際を書く
 - ここからはVisual studioを使う

Step 4: コンパイル

- デバック作業(ビルド⇒エラー確認⇒修正)
 - バグ(エラー)が無くなるまで続ける
- 完成したら実行

484

```

#include <iostream>
using namespace std;

void main(){
    int a, b, c; // 整数値A, B, C
    // 1. 整数値AとBを入力する
    cin >> a; cin >> b;
    // 2. A>0である限り以下の処理を繰り返す
    while (a>0) {
        // 2.1 B % A の値をCとする
        c = b % a;
        // 2.2 Aの値をBに代入する
        b = a;
        // 2.3 Cの値をAに代入する
        a = c;
    }
    // 3. Bの値を表示する
    cout << b;
}
  
```

main関数

485

Step 5: 確認(理解)と修正

- トレース表を書いて確認しよう!

486

Aを27, Bを90として
トレース表を作成せよ

＜Euclidの互除法＞

1. 整数値AとBを入力する
2. $A > 0$ である限り繰り返す
 - 2.1 $B \% A$ の値をCとする
 - 2.2 Aの値をBに代入する
 - 2.3 Cの値をAに代入する
3. Bの値を表示する

	A	B	C
1	27	90	—
2	27	90	—
2.1			
2.2			
2.3			
2			
2.1			
2.2			
2.3			
2.			
3.			

487

正解は・・・？

488

コンピュータの動きを理解しよう

Aを27, Bを90として
トレース表を作成せよ

Euclidの互除法

1. 整数値AとBを入力する
2. $A > 0$ である限り
以下の処理を繰り返す
 - 2.1 $B \% A$ の値をCとする
 - 2.2 Aの値をBに代入する
 - 2.3 Cの値をAに代入する
3. Bの値を表示する

	A	B	C
1	27	90	—
2	27	90	—
2.1	27	90	9
2.2	27	27	9
2.3	9	27	9
2	9	27	9
2.1	9	27	0
2.2	9	9	0
2.3	0	9	0
2.	0	9	0
3.	0	9	0

489

プログラミングの基本的な心構え

- 分かり易いプログラムの作成を心掛ける
 - プログラムが冗長になってもよい
- 使用する機能は十分理解する
 - 理解していない機能は、バグの原因
- コンパイラの警告メッセージは無視しない
 - 理由を考え、警告を消すように努力する
- 正当な理由がある場合にのみ修正する
 - 試しに変更(後から動作確認)はダメ

490

コンパイルエラーについて

- 正しいプログラムを書くこと
 - プログラムは一字でも間違えてはいけない
 - コンピュータはプログラムを忠実に実行する
(たとえプログラムが間違っている場合でも・・・)
- エラー(警告)メッセージはじっくり読む
 - バグを特定するヒントが含まれている
 - 最初のエラー(警告)から順番に対処する
 - コンパイル時にバグを除去しておけば、
実行時のエラーを減らせる

491

クロージングテスト

今回、習ったことを復習するために、
クロージングテストを行います

問題ごとに動画を止めて、答えをメモしましょう。

最後に、答えを動画で表示しますので、
自分が理解しているか、確認しましょう。

クロージングテスト

【問1】より古くから利用されてきたのは？

【問2】構造化されたアルゴリズム

しか書けないのは？

【問3】より多様な書き方ができる

(自由度が高い)のは？

【問4】この授業で主に用いているのは？

<選択肢> ①フローチャート, ②NSチャート

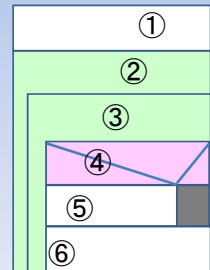
493

【問5】以下のNSチャートで必要ないのは？

【問6】間違っている場所を修正する場合、
修正すべき位置に近い場所は？

```
while(s != 1){
    for(p = 0; p <= s-2; p++){
        if(a[p] > a[p+1]){
            t = a[p+1];
            a[p+1] = a[p];
            a[p] = t;
        }
    }
    sortedPos--;
}
```

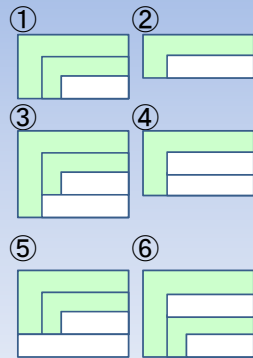
<選択肢>



クロージングテスト

```
void main(){
    for(a=1; a < 10; a++){
        for(b=1; b < 10; b++){
            cout<<a <<" * "<< b
                <<"=" << a*b;
        }
        cout << endl;
    }
}
```

【問7】プログラムを
示すものを選びなさい



答え合わせ

全ての答えをメモしましたか？

解答

【1】 ①

【2】 ②

【3】 ①

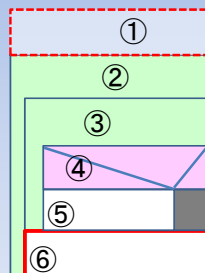
【4】 ②

【5】 ①

【6】 ⑥

【7】 ③

<問5 / 6>



お疲れさまでした

